

Time Complexity Analysis - MATDISC

G163

1210527 - Diogo Moreira
1230684 - Francisco Oliveira
1231205 - Inês Oliveira
1220804 - Susana Loureiro

June 2024

1 Theoretical Framework

In algorithm analysis, it's crucial to understand the efficiency of an algorithm concerning the size of its input. One commonly used measure for this purpose is Big O notation.

Big O notation provides an upper bound on the growth rate of a function, indicating how the runtime or space requirements of an algorithm scale concerning the size of the input. It describes the worst-case scenario, representing the maximum time or space required by an algorithm as the input size approaches infinity.

For example, if an algorithm's time complexity is $O(n^2)$, it means that the algorithm's runtime increases quadratically with the size of the input (n). Similarly, if the space complexity is $O(n)$, it indicates that the algorithm's memory usage grows linearly with the input size.

Understanding Big O notation helps in comparing the efficiency of different algorithms and predicting how they will perform as the input size increases.

2 US13

```
function numberOfVertices(edges):  
    vertices = empty list  
    for each edge in edges:  
        value1 = false  
        value2 = false  
        x1 = edge.getP1()  
        x2 = edge.getP2()  
        for each point in vertices:  
            if x1 == point:  
                value1 = true
```

```

        if x2 == point:
            value2 = true
    if not value1:
        append x1 to vertices
    if not value2:
        append x2 to vertices
    return vertices

function sortArrayListPrimitivePerPrice(edges):
    for i from 0 to size(edges) - 1:
        for j from i + 1 to size(edges):
            if edges[j].getPrice() < edges[i].getPrice():
                swap edges[i] and edges[j]

function getVertices(edges):
    vertices = empty list
    for each edge in edges:
        if edge.getP1() not in vertices:
            append edge.getP1() to vertices
        if edge.getP2() not in vertices:
            append edge.getP2() to vertices
    return vertices

function find(parent, i):
    if parent[i] != i:
        parent[i] = find(parent, parent[i])
    return parent[i]

function union(parent, rank, x, y):
    rootX = find(parent, x)
    rootY = find(parent, y)
    if rootX != rootY:
        if rank[rootX] > rank[rootY]:
            parent[rootY] = rootX
        else if rank[rootX] < rank[rootY]:
            parent[rootX] = rootY
        else:
            parent[rootY] = rootX
            rank[rootX] += 1

function kruskalAlgorithm(edges):
    result = empty list
    vertices = getVertices(edges)
    V = size(vertices)
    parent = new array of size V
    rank = new array of size V
    for i from 0 to V:
        parent[i] = i
        rank[i] = 0
    for each edge in edges:

```

```

    x = findVertexIndex(edge.getP1(), vertices)
    y = findVertexIndex(edge.getP2(), vertices)
    xRoot = find(parent, x)
    yRoot = find(parent, y)
    if xRoot != yRoot:
        append edge to result
        union(parent, rank, xRoot, yRoot)
    return result

function findVertexIndex(p, vertices):
    for i from 0 to size(vertices):
        if vertices[i] == p:
            return i
    return -1

```

2.1 Analysis - US13

Being n the number of edges and k the number of vertices,

function numberOfVertices

vertices = empty list	1A
for each edge in edges:	$n + 1$ C
value1 = false	n A
value2 = false	n A
x1 = edge.getP1()	n A
x2 = edge.getP2()	n A
for each point in vertices:	$n(k + 1)$ C
if x1 == point then value1 = true	$n(k + 1)$ A
if x2 == point:	$n(k + 1)$ A
value2 = true	$n(k + 1)$ A
if not value1:	$n(k + 1)$ A
append x1 to vertices	$n(k + 1)$ A
if not value2:	$n(k + 1)$ A
append x2 to vertices	$n(k + 1)$ A
return vertices	1R

- The first loop (**for each edge in edges**) makes $n+1$ comparisons, where n is the number of edges. Therefore, its complexity is $O(n)$.
- The second loop (**for each point in vertices**) makes $n(k + 1)$ comparisons, where k is the number of points. Therefore, its complexity is $O(n * k)$.

Thus, the total time complexity of this function is $O(n * k)$.

function sortArrayPrimitivePerPrice

vertices = empty list	1A
for i from 0 to size(edges) - 1	n-1C
for j from i + 1 to size(edges)	$\frac{n \cdot (n-1)}{2}$ C
if edges[j].getPrice() > edges[i].getPrice()	$\frac{n \cdot (n-1)}{2}$ A
swap edges[i] and edges[j]	n(n-1)/2A

- The first loop (for i from 0 to size(edges) - 1) makes $n - 1$ comparisons, yielding a time complexity of $O(n)$
- The second loop (for j from i+1 to size(edges)) makes $\frac{n \cdot (n-1)}{2}$ comparisons, resulting in a time complexity of $O(n^2)$.

Thus, the total time complexity of this function is $O(n^2)$.

function getVertices

vertices = empty list	1A
for each edge in edges:	n+1C
if edge.getP1() not in vertices:	n*kA
append edge.getP1() to vertices	n*kA
if edge.getP2() not in vertices:	n*kA
append edge.getP2() to vertices	n*kA
return vertices	1R

- The first loop (for each edge in edges) makes $n + 1$ comparisons, resulting in a time complexity of $O(n)$.
- The following if conditions make $n * k$ attributions, resulting in a time complexity of $O(n*k)$.

Therefore, the total time complexity of this function is $O(n * k)$.

function find

if parent[i] != i:	1A
parent[i] = find(parent, parent[i])	log nA
return parent[i]	1R

This function only makes attributions, however, since it is called most times, its time complexity depends on when is being called, therefore the time complexity is $O(\log n)$.

function union

rootX = find(parent, x)	lognA
rootY = find(parent, y)	lognA
if rootX != rootY:	lognA
if rank[rootX] < rank[rootY]:	lognC
parent[rootY] = rootX	lognC
else if rank[rootX] > rank[rootY]:	lognC
parent[rootX] = rootY	lognA
else:	
parent[rootY] = rootX	lognA
rank[rootX] += 1	lognA

This function only makes attributions, however, since it is called most times, its time complexity depends on when is being called, therefore the time complexity is $O(\log n)$.

function kruskalAlgorithm

result = empty list	1A
vertices = getVertices(edges)	1A
V = size(vertices)	1A
parent = new array of size V	1A
rank = new array of size V	1A
for i from 0 to V:	n+1C
parent[i] = i	nA
rank[i] = 0	nA
for each edge in edges:	k+1C
x = findVertexIndex(edge.getP1(), vertices)	kA
y = findVertexIndex(edge.getP2(), vertices)	kA
xRoot = find(parent, x)	k*lognA
yRoot = find(parent, y)	k*lognA
if xRoot != yRoot:	kA
append edge to result	kA
union(parent, rank, xRoot, yRoot)	kA
return result	1R

- The first loop makes $n + 1$ comparisons, resulting in a time complexity of $O(n)$.
- The second loop makes $k + 1$ comparisons, yielding a time complexity of $O(k)$.
- When the find and union operations are made, they have a time complexity of $O(k \cdot \log n)$.

Therefore, the total time complexity of this algorithm is $O(k \cdot \log n)$.

function findVertexIndex

for i from 0 to size(vertices):	n+1C
if vertices[i] == p:	nA
return i	1R
return -1	1R

- The loop (**for i from 0 to size(vertices)**) makes $n + 1$ comparisons, where n is the number of vertices, resulting in a time complexity of $O(n)$.
- The following **if** condition is executed n times, leading to a time complexity of $O(n)$.

Therefore, its time complexity is $O(n)$.

Given all of these functions, the time complexity of this implementation of the Kruskal Algorithm is $O(k \cdot \log n)$. This notation indicates that the algorithm's runtime grows logarithmically with the number of vertices and linearly with the number of edges.

3 US17

```

function indexOfMin(costsOfVertices, verticesVisited, numVertices):
    min = Infinity
    indexOfMinimum = -1
    for j from 0 to numVertices - 1:
        if not verticesVisited[j] and costsOfVertices[j] < min:
            min = costsOfVertices[j]
            indexOfMinimum = j
    return indexOfMinimum

function dijkstraAlgorithm(edges, startingPoint, endPoint):

```

```

resultPath = empty list
vertices = numberOfVertices(edges)
numVertices = size of vertices list

costsOfVertices = list of size numVertices, initialized with
    Infinity for all elements
verticesUsed = list of size numVertices, initialized with false for
    all elements
pathVertices = list of size numVertices, initialized with null for
    all elements

indexOfStartingPoint = index of startingPoint in vertices list
indexOfEndPoint = index of endPoint in vertices list

costsOfVertices[indexOfStartingPoint] = 0

for i from 0 to numVertices - 2:
    indexOfMinimum = indexofMin(costsOfVertices, verticesUsed,
        numVertices)
    if indexOfMinimum is -1 or indexOfMinimum is indexOfEndPoint:
        break

    chosenVertex = vertices[indexOfMinimum]
    verticesUsed[indexOfMinimum] = true

    for each edge in edges:
        if edge.p1 is equal to chosenVertex:
            indexOfNext = index of edge.p2 in vertices list
            if not verticesUsed[indexOfNext]:
                cost = costsOfVertices[indexOfMinimum] + edge.price
                if cost < costsOfVertices[indexOfNext]:
                    costsOfVertices[indexOfNext] = cost
                    pathVertices[indexOfNext] = chosenVertex

currentPoint = endPoint
while currentPoint is not null and currentPoint is not equal to
    startingPoint:
        currentIndex = searchForIndexPoint(vertices,currentPoint)
        beforePoint = pathVertices[currentIndex]
        if beforePoint is null:
            break
        cost = costsOfVertices[currentIndex] - costsOfVertices[index of
            beforePoint in vertices list]
        add Edge(beforePoint, currentPoint, cost) to resultPath
        currentPoint = beforePoint

return resultPath

function searchForIndexPoint(points, point):
    For i from 0 to size of points - 1:

```

```

    If points[i] equals point:
        Return i
Return -1

```

3.1 Analysis - US17

Being n the number of vertices and k the number of edges.

function indexOfMin

min = INFINITY	1A
indexOfMinimum = -1	1A
for j = 0 to numVertices-1:	nC
if not verticesVisited[j] and costsOfVertices[j] < min then min = costsOfVertices[j] and indexOfMinimum = j	nC + nA
return indexOfMinimum	1R

- The loop (for j from 0 to numVertices - 1) makes n comparisons, resulting in a time complexity of $O(n)$.
- The condition if (if not verticesVisited[j] and costsOfVertices[j] < min) involves $n - 1$ attributions, yielding a time complexity of $O(n)$.

Therefore, the total time complexity of this function is $O(n)$.

function dijkstraAlgorithm

resultPath = empty list of Edge	1A
vertices = numberOfVertices(edges)	1A
numVertices = size of vertices	1A
costsOfVertices = list of size numVertices with all elements set to INFINITY	1A
verticesUsed = list of size numVertices with all elements set to false	1A
pathVertices = list of size numVertices with all elements set to null	1A
indexOfStartingPoint = index of startingPoint in vertices	1A
indexOfEndPoint = index of endPoint in vertices	1A

costsOfVertices[indexOfStartingPoint] = 0	1A
for i = 0 to numVertices-2:	n-1C
indexOfMinimum = indexOfMin(costsOfVertices, verticesUsed, numVertices)	nA
if indexOfMinimum == -1 or indexOfMinimum == indexOfEndPoint then break	nC
chosenVertice = vertices[indexOfMinimum]	nA
verticesUsed[indexOfMinimum] = true	nA
for edge in edges:	n(k+1)C
if edge.p1 equals chosenVertice then indexOfNext = index of edge.p2 in vertices	(n-2)(n*k(+1))A
if not verticesUsed[indexOfNext] then cost = costsOfVertices[indexOfMinimum] + edge.price	(n-2)(n*k(+1))A
if cost < costsOfVertices[indexOfNext] then costsOfVertices[indexOfNext] = cost and pathVertices[indexOfNext] = chosenVertice and currentPoint = endPoint	(n-2)(n*k(+1))A
while currentPoint is not null and currentPoint does not equal startingPoint:	log nC
beforePoint = pathVertices[indexOf(currentPoint in vertices)]	log nC
if beforePoint is null then break	log nC
cost = costsOfVertices[indexOf(currentPoint in vertices)] - costsOfVertices[indexOf(beforePoint in vertices)]	log n
resultPath.add(new Edge(beforePoint, currentPoint, cost))	log n

currentPoint = beforePoint	$\log n$
return resultPath	1R

- The first loop (**for each edge in edges**) makes $n+1$ comparisons, where n is the number of edges. Therefore, its complexity is $O(n)$.
- The second loop (**for each point in vertices**) makes $n(k+1)$ comparisons, where k is the number of points. Therefore, its complexity is $O(n * k)$.

Thus, the total time complexity of this function is $O(n * k)$.

function searchForIndexPoint

For i from 0 to size of points - 1:	nC
If points[i] equals point:	n-1A
Return i	1R
Return -1	1R

- The first loop (**for i from 0 to size of points - 1**) makes n comparisons, resulting in a time complexity of $O(n)$.
- The condition **if** within the loop involves $n - 1$ comparisons, leading to a time complexity of $O(n)$.

Therefore, the time complexity of this function is $O(n)$.

Considering the time complexities of each function, the worst-case time complexity of the Dijkstra Algorithm implementation is $O(n * k)$, where n is the number of vertices and k is the number of edges. This notation implies that the algorithm's runtime grows linearly with the number of vertices and edges.

4 US18

The procedure implemented for this US includes the algorithm implemented in the US17 along with two more functions to find the closest Assembly Point.

```
function comparingResults(results):
    path = empty list
    cost = 0
    costs = empty list

    for each array in results:
```

```

    for each edge in array:
        cost += edge.price
    add cost to costs
    cost = 0

minCost = costs[0]
posMinCost = 0
for each i in costs:
    if i < minCost:
        minCost = i
        posMinCost = searchForIndexCost(costs, minCost)

path = results[posMinCost]
return path

function searchForIndexCost(costs, cost):
    for i from 0 to size of costs - 1:
        if costs[i] equals cost:
            return i
    return -1

```

4.1 Analysis - US18

Being n the number of vertices, k the number of edges and l the number of costs existent

function comparingResults

path = Empty ArrayList of Edges	1A
cost = 0	1A
costs = Empty ArrayList of Integers	1A
For each array in results:	$n + 1C$
For each edge e in array:	$(n + 1) * (k + 1)C$
cost += $e.getPrice()$	$n * kA$
Add cost to costs	$n * kA$
cost = 0	$n * kA$
minCost = costs[0]	1A
posMinCost = 0	1A
For each integer i in costs:	$(l + 1)C$
If i is less than minCost:	lC
minCost = i	lA
posMinCost = searchForIndexCost(costs, minCost)	lA
path = results[posMinCost]	1A
Return path	1A

- The first loop (for each array in results) makes $n+1$ comparisons, resulting in a time complexity of $O(n)$.
- The second loop (for each edge in array) makes $l+1$ comparisons, yielding a time complexity of $O(l)$.
- The **if** condition involves l assignments, leading to a time complexity of $O(l)$.

Therefore, the time complexity of this function is $O(n) + O(l)$.

function searchForIndexCost

For i from 0 to size of costs - 1:	lC
If costs[i] equals cost:	$l - 1A$
Return i	$1R$
Return -1	$1R$

- The first loop (for i from 0 to size of points - 1) makes l comparisons, resulting in a time complexity of $O(l)$.
- The **if** condition involves $l - 1$ comparisons, leading to a time complexity of $O(l)$.

Therefore the time complexity of this function is $O(l)$.

The overall time complexity of the procedure is $O(n^2)$, where n is the number of vertices. This indicates a quadratic increase in runtime as the number of vertices grows.

The quadratic complexity stems from nested loops in the ‘comparingResults’ function, iterating over each array and each edge. Additionally, the ‘searchForIndexCost’ function, called within ‘comparingResults’, contributes a linear $O(n)$ complexity.

In summary, the $O(n^2)$ complexity underscores scalability concerns as input size increases.